Projeto Interdisciplinar de Sistemas de Informação 2 (PISI 2)

FLYFOOD: UM ALGORITMO DE FORÇA BRUTA PARA OTIMIZAÇÃO DE ROTAS DE ENTREGA POR DRONE BASEADO NA DISTÂNCIA DE MANHATTAN

Júlia Galindo · Davi Guaraná · Eloísa Fernanda · Gabriel Soares

Curso de Sistemas de Informação — Disciplina de Projeto Interdisciplinar de Sistemas de Informação 2 (PISI 2)

Resumo

O crescimento da entrega de mercadorias por meio de veículos aéreos não tripulados (drones) tem impulsionado a necessidade de algoritmos eficientes de roteamento. Este trabalho apresenta o FlyFood, um sistema que calcula a rota mais eficiente para um drone realizar entregas em uma cidade representada por uma matriz, partindo de um ponto de referência (R), visitando todos os pontos de entrega e retornando ao ponto de origem, sempre se deslocando apenas na horizontal ou na vertical. O problema é modelado como uma variação do Problema do Caixeiro-Viajante (PCV) e resolvido por meio de uma abordagem de força bruta, que gera todas as permutações possíveis dos pontos de entrega e seleciona aquela de menor custo, calculado pela distância de Manhattan. São descritos o processo de leitura do mundo (matriz da cidade), o algoritmo de cálculo do percurso ótimo e a complexidade computacional da solução. Testes empíricos com 10, 11 e 12 pontos de entrega confirmam o crescimento fatorial previsto teoricamente, evidenciando as limitações práticas da abordagem e possíveis extensões futuras.

Palavras-chave: *Roteamento de drones; Problema do Caixeiro-Viajante; Distância de Manhattan; Força bruta; Otimização combinatória.*

1 Introdução

A utilização de drones para entrega de mercadorias vem se consolidando como uma alternativa aos meios tradicionais de logística, sobretudo em cenários urbanos onde a agilidade e a redução de custos operacionais são fatores determinantes. Um dos principais desafios associados a essa modalidade de entrega é a definição da rota que o drone deve percorrer para atender a todos os pontos de destino com o menor custo possível, seja em distância, tempo ou consumo de energia.

Esse tipo de problema é uma instância do clássico Problema do Caixeiro-Viajante (PCV), amplamente estudado na literatura de otimização combinatória, cuja característica principal é o crescimento fatorial do espaço de soluções conforme o número de pontos a visitar aumenta. No contexto deste trabalho, o ambiente de deslocamento do drone é simplificado para uma malha (grade), na qual os movimentos permitidos são restritos às direções horizontal e vertical, o que torna a distância de Manhattan a métrica adequada para o cálculo do custo entre dois pontos.

O projeto FlyFood, desenvolvido para a disciplina de Projeto Interdisciplinar de Sistemas de Informação 2 (PISI 2), tem como objetivo implementar um sistema capaz de ler a representação de uma cidade a partir de um arquivo de texto, identificar o ponto de referência do drone e os pontos de entrega, e calcular a rota de menor custo que parte do ponto de referência, visita todas as entregas e retorna à origem. Este relatório descreve a fundamentação do problema, a metodologia empregada na implementação, a análise de complexidade da solução e as considerações finais sobre o trabalho desenvolvido.

2 Fundamentação Teórica

O Problema do Caixeiro-Viajante consiste em, dado um conjunto de cidades e as distâncias entre cada par delas, encontrar a rota de menor custo que visita cada cidade exatamente uma vez e retorna à cidade de origem [1]. Trata-se de um problema classificado como NP-difícil, o que significa que, para instâncias grandes, não se conhece um algoritmo capaz de encontrar a solução ótima em tempo polinomial [2].

Quando o deslocamento entre pontos ocorre exclusivamente nas direções horizontal e vertical — como é o caso de um drone que se move sobre uma grade que representa ruas ou quarteirões de uma cidade — a métrica de distância mais adequada é a distância de Manhattan (ou distância L1), definida para dois pontos p = (x1, y1) e q = (x2, y2) como:

$$d(p, q) = |x1 - x2| + |y1 - y2|$$

Essa métrica é amplamente utilizada em problemas de roteamento urbano e em grades regulares, por refletir de forma mais fiel o custo real de deslocamento em ambientes onde não é possível o movimento diagonal [3].

3 Metodologia

O sistema FlyFood foi implementado em Python e é composto por dois módulos principais: a leitura e interpretação do mundo (cidade) e o cálculo do percurso ótimo do drone.

3.1 Representação e leitura do mundo

A cidade é representada por uma matriz descrita em um arquivo de texto, na qual cada célula pode conter um ponto de referência (R), um ponto de entrega ou um espaço vazio. A função ler_mundo(arquivo) é responsável por interpretar esse arquivo e convertê-lo em uma lista de pontos utilizável pelo restante do programa. O processo segue as seguintes etapas:

- Abertura do arquivo e leitura da primeira linha, que define as dimensões da matriz;
- Percurso de cada uma das linhas seguintes da matriz;
- Verificação do valor de cada célula, identificando o ponto de referência e os pontos de entrega;
- Geração de uma lista com todas as tuplas de pontos encontrados (nome e coordenadas);
- Tratamento de erros de leitura, de modo que o programa não seja interrompido caso o arquivo esteja mal formatado.

3.2 Cálculo da melhor rota

A função obter_percurso é responsável por calcular a rota de menor custo entre o ponto de referência e os pontos de entrega, empregando uma estratégia de força bruta. O algoritmo segue os passos descritos a seguir:

1. O ponto de referência (R) é separado dos demais pontos de entrega;
2. Todas as possíveis ordens de visita aos pontos de entrega são geradas por meio da função permutations da biblioteca padrão de Python;
3. Para cada ordem gerada, o custo total do percurso é calculado somando-se as distâncias de Manhattan entre pontos consecutivos;
4. Cada percurso avaliado sempre inicia em R, passa por todos os pontos de entrega e retorna a R;

5. A cada permutação avaliada, o custo obtido é comparado com o menor custo encontrado até o momento, atualizando-se a melhor rota quando aplicável;
6. Ao final da busca, o algoritmo retorna a sequência de nomes dos pontos que compõem a rota de menor custo, na ordem em que devem ser visitados.

3.3 Análise de complexidade

Por se tratar de uma abordagem exaustiva, o algoritmo avalia todas as n! permutações possíveis dos n pontos de entrega, resultando em uma complexidade de tempo O(n!). Essa característica garante que a solução encontrada seja sempre ótima (a rota de menor custo global), mas limita a aplicabilidade prática do algoritmo a instâncias com um número reduzido de pontos de entrega, uma vez que o custo computacional cresce de forma extremamente rápida com o aumento de n. Para cenários com maior número de entregas, técnicas heurísticas ou metaheurísticas (como algoritmos genéticos, colônia de formigas ou 2-opt) tornam-se alternativas mais adequadas, embora não garantam a solução ótima.

4 Resultados e Discussão

A implementação do FlyFood permite, a partir de um arquivo de entrada representando a matriz da cidade, obter de forma determinística a rota ótima que o drone deve seguir para realizar todas as entregas com o menor custo total de deslocamento. Por garantir a exploração completa do espaço de soluções, o algoritmo é adequado como referência de corretude (baseline) para comparação com abordagens heurísticas em trabalhos futuros.

4.1 Testes de desempenho

Para verificar empiricamente o comportamento fatorial previsto na análise de complexidade (Seção 3.3), foram cronometrados os tempos de execução do algoritmo para instâncias com 10, 11 e 12 pontos de entrega. Os testes foram realizados em um notebook equipado com processador Intel Core i3 100U (4,7 GHz, 8 núcleos), com outros programas em execução em segundo plano, o que pode ter introduzido pequenas oscilações nos tempos medidos. Os resultados obtidos são apresentados na Tabela 1.

Pontos de entrega (n)	Tempo medido	n! / 10! (crescimento teórico esperado)
10	≈ 2 s	1× (referência)
11	≈ 24 s	11× → 22 s esperados
12	≈ 5 min 20 s (320 s)	132× → 264 s esperados

Tabela 1: Tempo de execução medido do FlyFood para 10, 11 e 12 pontos de entrega, em comparação com o crescimento fatorial teórico esperado tomando n = 10 como referência.

Os dados confirmam de forma consistente o comportamento fatorial previsto teoricamente: ao multiplicar o tempo obtido para 10 pontos (2 s) pelo fator 11, chega-se a aproximadamente 22 s, valor muito próximo dos 24 s efetivamente cronometrados para 11 pontos. De forma semelhante, ao multiplicar o tempo de 11 pontos (24 s) pelo fator 12, obtêm-se aproximadamente 288 s, o que corresponde a cerca de 90% dos 320 s (5 min 20 s) medidos para 12 pontos. As pequenas divergências entre os valores teóricos e os medidos são esperadas e compatíveis com a execução de outros processos em segundo plano durante os testes, não comprometendo a identificação clara do padrão de crescimento fatorial O(n!) do algoritmo.

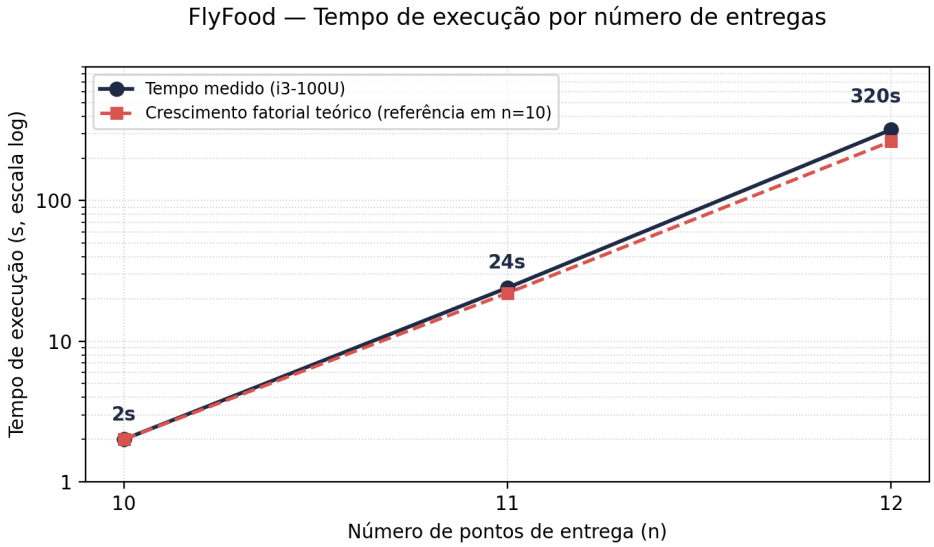


Figura 1: Tempo de execução medido (escala logarítmica) para 10, 11 e 12 pontos de entrega, comparado ao crescimento fatorial teórico esperado.

Como principal limitação observada, destaca-se o crescimento fatorial do tempo de execução à medida que o número de pontos de entrega aumenta, o que restringe o uso da força bruta a cenários de pequeno porte: o salto de apenas dois pontos adicionais (de 10 para 12) foi suficiente para elevar o tempo de execução de segundos para minutos. Esse comportamento reforça a necessidade de abordagens heurísticas ou metaheurísticas para instâncias maiores, conforme discutido na Seção 3.3.

5 Conclusão

Este trabalho apresentou o FlyFood, um sistema para cálculo da rota ótima de um drone de entregas em um ambiente representado por uma grade, utilizando a distância de Manhattan como métrica de custo e uma estratégia de força bruta baseada em permutações para a busca da solução ótima. A abordagem adotada garante a otimalidade da rota encontrada, servindo como base sólida para o entendimento do problema e para o desenvolvimento futuro de soluções mais escaláveis, capazes de lidar com um número maior de pontos de entrega por meio de heurísticas e metaheurísticas.

Divisão de Responsabilidades

Integrante	Responsabilidade
Júlia Galindo	Implementação da função ler_mundo
Davi Guaraná	Implementação da função obter_percurso (cálculo da melhor rota)
Eloísa Fernanda	Redação do README e do relatório final
Gabriel Soares	Organização e distribuição das tarefas entre os integrantes

Tabela 2: Divisão de responsabilidades entre os integrantes do grupo no desenvolvimento do projeto FlyFood.

Referências

[1] Applegate, D. L.; Bixby, R. E.; Chvátal, V.; Cook, W. J. The Traveling Salesman Problem: A Computational Study. Princeton University Press, 2006.

[2] Garey, M. R.; Johnson, D. S. Computers and Intractability: A Guide to the Theory of NP-Completeness. W. H. Freeman, 1979.

[3] Krause, E. F. Taxicab Geometry: An Adventure in Non-Euclidean Geometry. Dover Publications, 1986.

[4] Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; Stein, C. Introduction to Algorithms. 3ª ed. MIT Press, 2009.

Projeto acadêmico desenvolvido para fins educacionais — Disciplina de Projeto Interdisciplinar de Sistemas de Informação 2 (PISI 2).